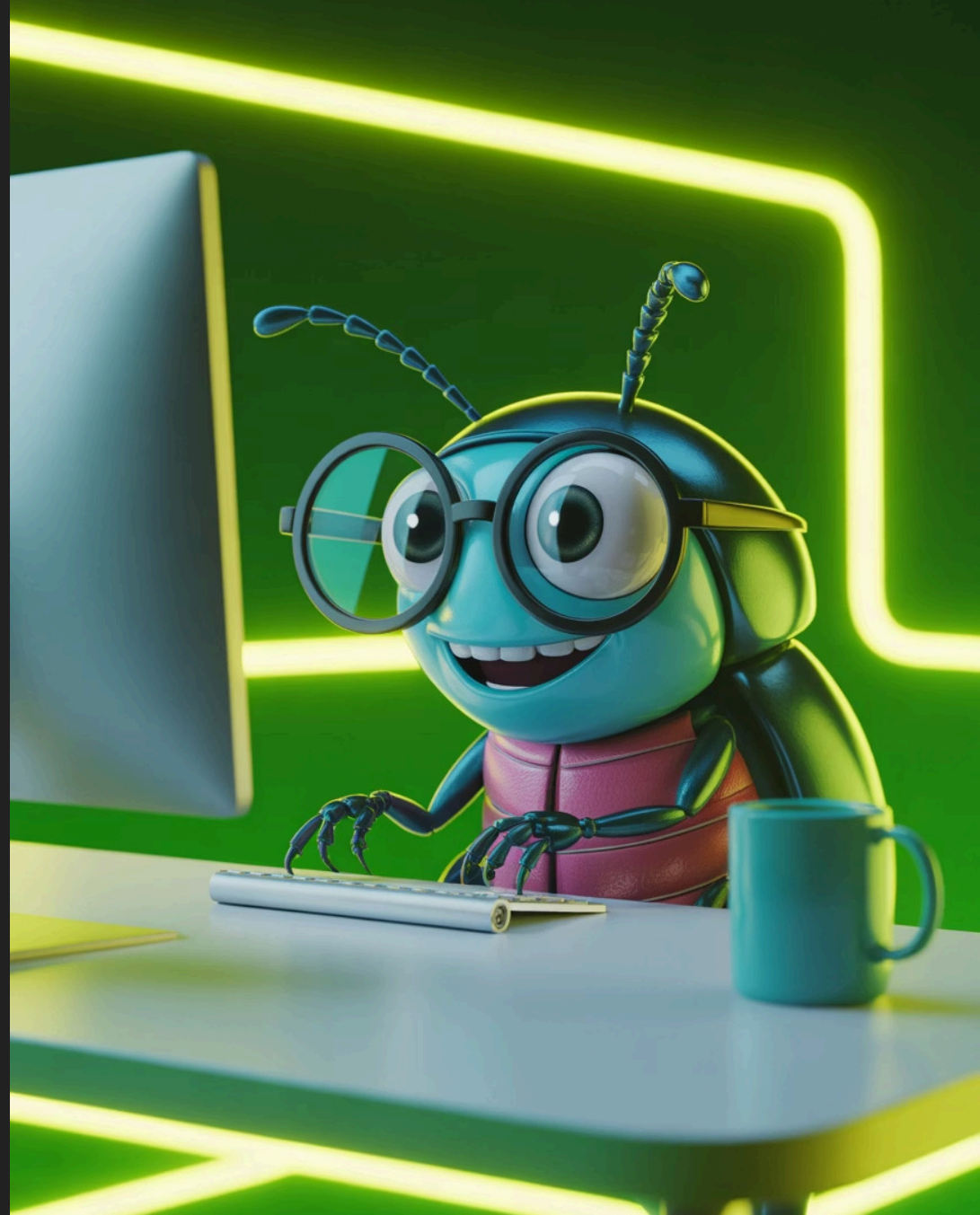


More Than Just assertEquals

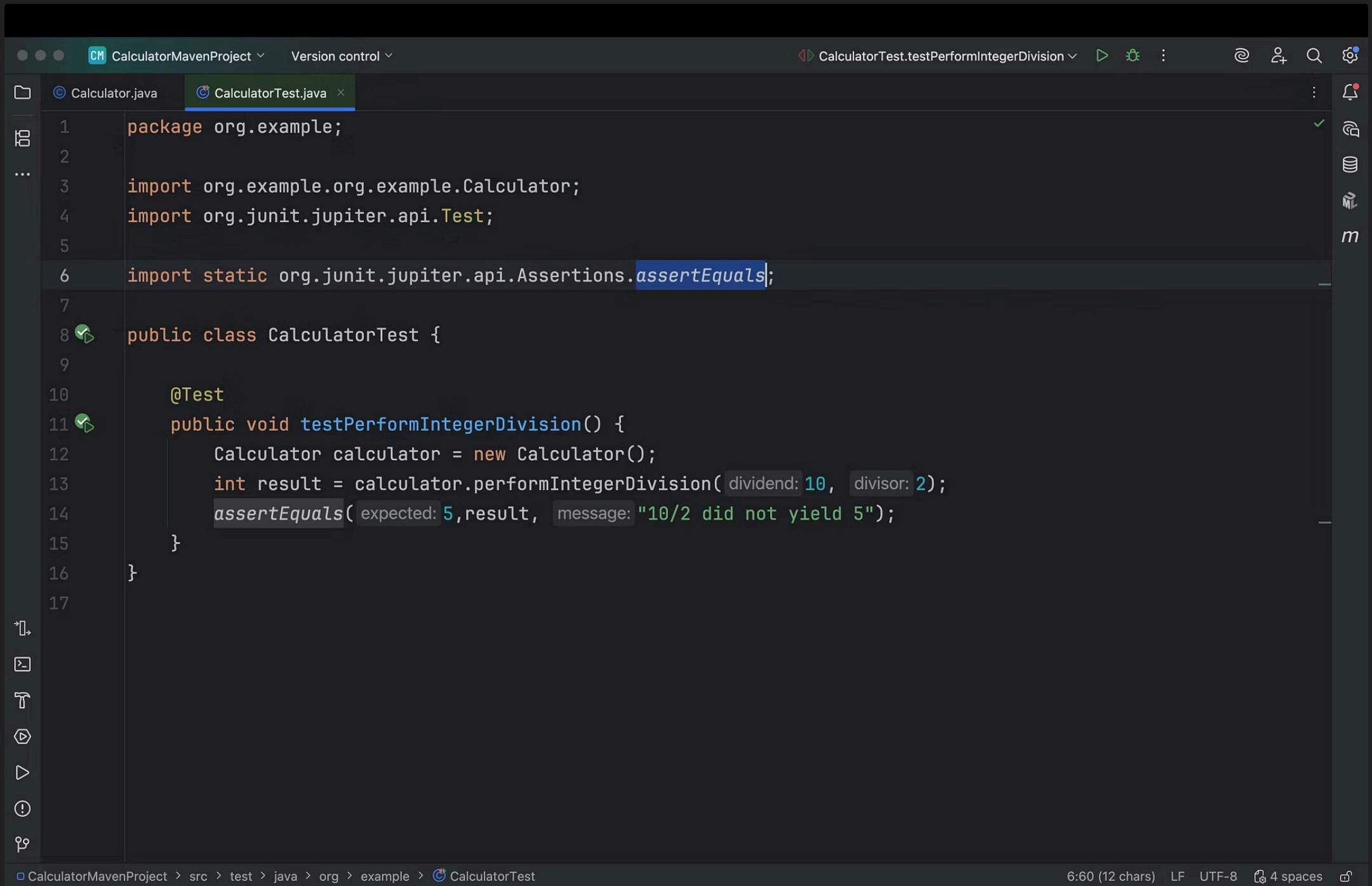
JUnit provides many assertions beyond
assertEquals to validate different behaviors

 by Rodrigo Martins Pagliares

Last update: 11/25/2025



The Method `Assertions.assertEquals`



The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The active file is 'CalculatorTest.java'. The code defines a 'Calculator' class and a 'CalculatorTest' class. The 'testPerformIntegerDivision' method in 'CalculatorTest' uses 'assertEquals' to verify the result of a division operation. The IDE interface includes a top bar with project and file names, a left sidebar with file explorer and tool windows, and a bottom status bar showing file encoding and line count.

```
1 package org.example;
2
3 import org.example.org.example.Calculator;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.assertEquals;
7
8 public class CalculatorTest {
9
10     @Test
11     public void testPerformIntegerDivision() {
12         Calculator calculator = new Calculator();
13         int result = calculator.performIntegerDivision(dividend: 10, divisor: 2);
14         assertEquals(expected: 5, result, message: "10/2 did not yield 5");
15     }
16 }
17
```

CalculatorMavenProject > src > test > java > org > example > CalculatorTest

6:60 (12 chars) LF UTF-8 4 spaces

The Method `Assertions.assertEquals` (contd.)

The `assertEquals` method is **overloaded** with several possible parameters **combination**.

```
1 package org.example;
2
3 import org.example.org.example.Calculator;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.a;
7
8 public class CalculatorTest {
9
10     @Test
11     public void testPerformIntegerDivision() {
12         Calculator calculator = new Calculator();
13         int result = calculator.performIntegerDivision(10, 2);
14         assertEquals(expected: 5, result, "The result of 10 divided by 2 is not 5");
15     }
16 }
```

Run CalculatorTest.testPerformIntegerDivision

✓ CalculatorTest (org.example) 14 ms
✓ testPerformIntegerDivision() 14 ms

1 test passed 1 test total, 14 ms

Process finished with exit code 0

Library/Java/JavaVirtualMachines/temurin-20-jdk/content/bin/java ...

Assertions for **Inequality** and Boolean Conditions



`assertNotEquals`

Verifies that two values are *not* equal and fails otherwise

Useful when ensuring unexpected or forbidden results do not occur

`assertTrue`

Validates that a condition evaluates to true

`assertFalse`

Confirms that a condition evaluates to false

These assertions help express logic clearly and improve readability

Assertions for **Inequality** and Boolean Conditions (contd.)

assertNotEquals

The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The active file is 'CalculatorTest.java'. The code defines a package, imports 'org.example.org.example.Calculator' and 'org.junit.jupiter.api.Test', and then defines a class 'CalculatorTest' with a test method 'testPerformIntegerDivision()'. The test method calls 'calculator.performIntegerDivision()' and 'assertEquals(5, result, message)'. A dropdown menu is open, showing various 'assertNotEquals' methods for different data types (int, Byte, byte, char, Long, long, Float, Double). The 'Run' button is visible, and the output console shows 'Process finished with exit code 0'.

```
1 package org.example;
2
3 import org.example.org.example.Calculator;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.assertNotEquals;
7
8 public class CalculatorTest {
9
10     @Test
11     public void testPerformIntegerDivision() {
12         Calculator calculator = new Calculator();
13         int result = calculator.performIntegerDivision(10, 2);
14         assertEquals("expected: 5, result: " + result, result, 5);
15     }
16 }
```

Run CalculatorTest.testPerformIntegerDivision

✓ CalculatorTest (org.example) 14 ms
✓ testPerformIntegerDivision() 14 ms

Process finished with exit code 0

Library/Java/JavaVirtualMachines/temurin-25-jdk/contents/home/bin/java ...

Assertions for **Inequality** and Boolean Conditions (contd.)

assertTrue

The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The file 'CalculatorTest.java' is open, showing the following code:

```
1 package org.example;
2
3 import org.example.org.example.Calculator;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.assertTrue;
7
8 public class CalculatorTest {
9
10     @Test
11     public void testPerformIntegerDivision() {
12         Calculator calculator = new Calculator();
13         int result = calculator.performIntegerDivision(10, 2);
14         assertEquals(5, result, "10/2 did not yield 5");
15     }
16 }
```

A tooltip for the `assertTrue` method is visible, showing several overloads:

- `assertTrue(boolean condition)` void
- `assertTrue(BooleanSupplier booleanSupplier)` void
- `assertTrue(boolean condition, String message)` void
- `assertTrue(BooleanSupplier booleanSupplier, String message)` void
- `assertTrue(boolean condition, Supplier<String> messageSupplier)` void
- `assertTrue(BooleanSupplier booleanSupplier, Supplier<String> messageSupplier)` void

The bottom of the IDE shows the 'Run' tab for 'CalculatorTest.testPerformIntegerDivision'. The test passed successfully, with the output:

```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...
Process finished with exit code 0
```

The status bar at the bottom indicates the file path: 'CalculatorMavenProject > src > test > java > org > example > CalculatorTest' and the encoding: '6:57 LF UTF-8 4 spaces'.

Assertions for **Inequality** and Boolean Conditions (contd.)

`assertFalse`

The screenshot shows an IDE window for a project named "CalculatorMavenProject". The active file is "CalculatorTest.java". The code defines a package, imports, and a test class "CalculatorTest" with a test method "testPerformIntegerDivision". The test method uses "assertEquals" to verify the result of a division. The IDE shows a tooltip for "assertFalse" with several overloads. Below the code, the "Run" tab shows the test execution results: "1 test passed" and "1 test total, 14 ms". The output console shows the command used to run the test and the message "Process finished with exit code 0".

```
1 package org.example;
2
3 import org.example.org.example.Calculator;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.assertFalse; Tab to complete
7
8 public class CalculatorTest {
9
10     @Test
11     public void testPerformIntegerDivision() {
12         Calculator calculator = new Calculator();
13         int result = calculator.performIntegerDivision(10, 2);
14         assertEquals(5, result, "10/2 did not yield 5");
15     }
16 }
```

Run CalculatorTest.testPerformIntegerDivision

✓ CalculatorTest (org.example) 14 ms
✓ testPerformIntegerDivision() 14 ms

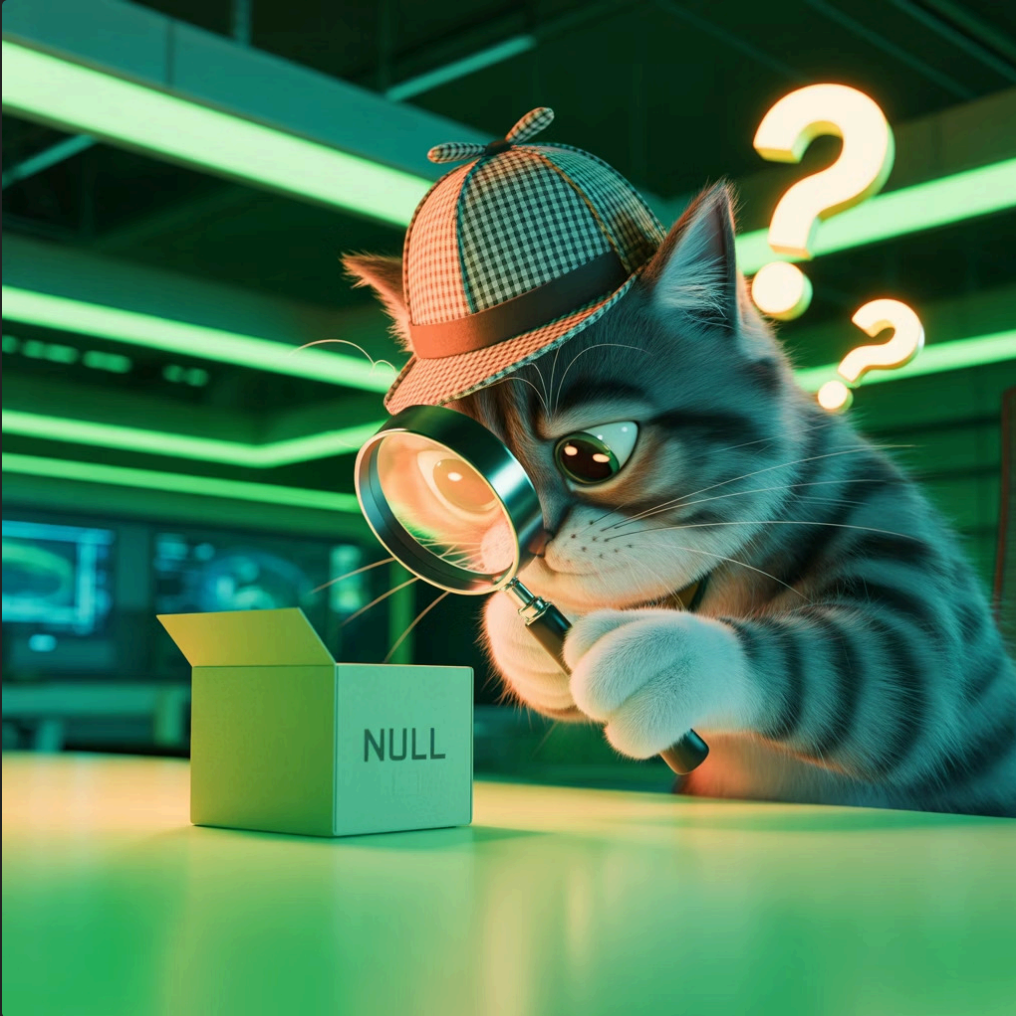
✓ 1 test passed 1 test total, 14 ms

/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...

Process finished with exit code 0

CalculatorMavenProject > src > test > java > org > example > CalculatorTest 6:55 LF UTF-8 4 spaces

Assertions for Nullability



`assertNull`

Checks that a returned object is null

`assertNotNull`

Ensures an object reference is not null

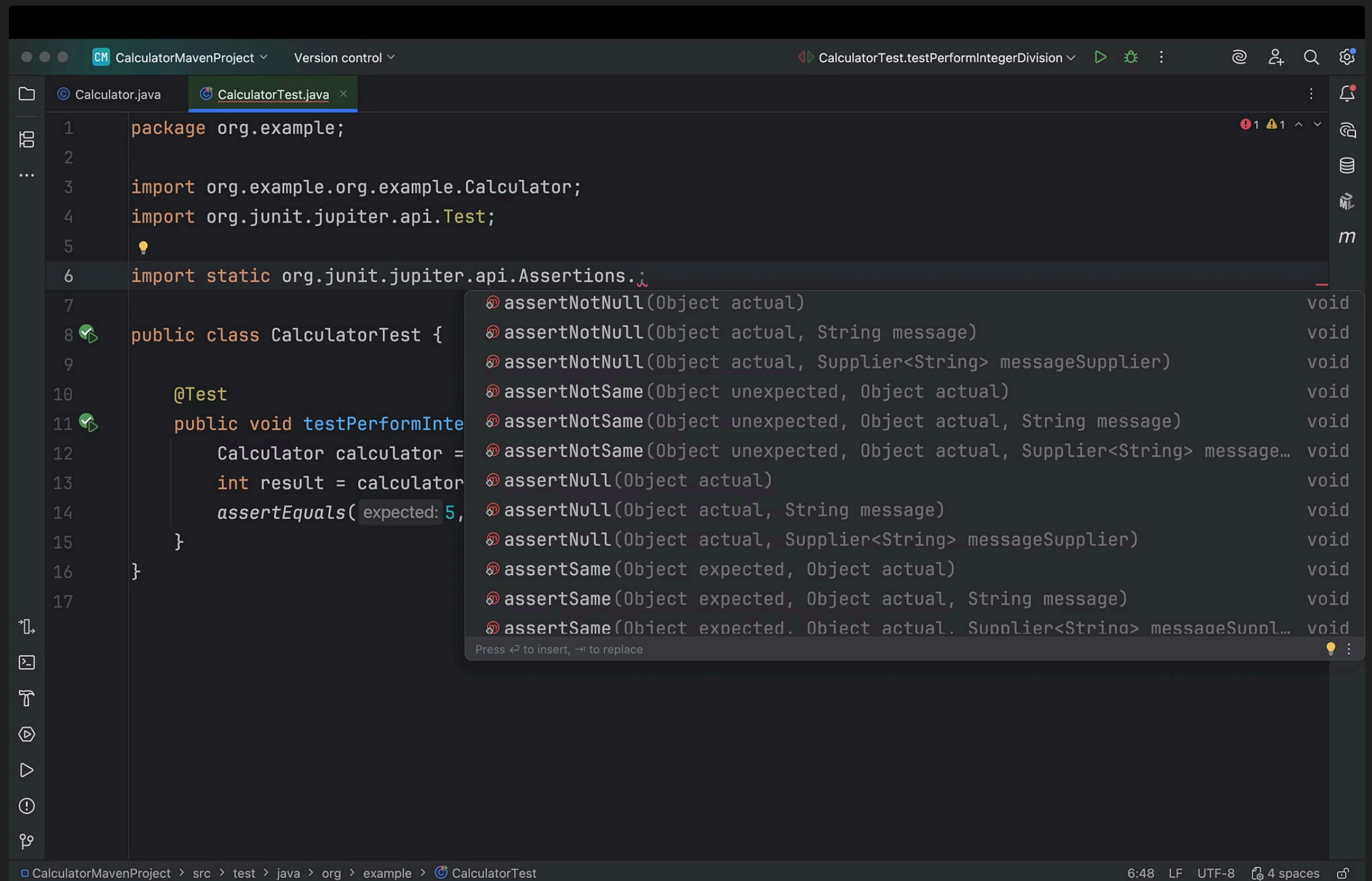
These assertions are helpful when testing factory methods or optional values

Nullability checks prevent unintended `NullPointerException` behavior

They clarify the expected lifecycle of returned objects

More Than Just assertEquals

assertNull, assertNotNull



The screenshot shows an IDE window for a project named "CalculatorMavenProject". The active file is "CalculatorTest.java". The code in the file is as follows:

```
1 package org.example;
2
3 import org.example.org.example.Calculator;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.*;
7
8 public class CalculatorTest {
9
10     @Test
11     public void testPerformIntegerDivision() {
12         Calculator calculator = new Calculator();
13         int result = calculator.performIntegerDivision(10, 2);
14         assertEquals(expected: 5, actual: result);
15     }
16 }
17
```

A dropdown menu is open, showing a list of JUnit assertions with their return types:

- `assertNotNull(Object actual)` void
- `assertNotNull(Object actual, String message)` void
- `assertNotNull(Object actual, Supplier<String> messageSupplier)` void
- `assertNotSame(Object unexpected, Object actual)` void
- `assertNotSame(Object unexpected, Object actual, String message)` void
- `assertNotSame(Object unexpected, Object actual, Supplier<String> messageSupplier)` void
- `assertNull(Object actual)` void
- `assertNull(Object actual, String message)` void
- `assertNull(Object actual, Supplier<String> messageSupplier)` void
- `assertSame(Object expected, Object actual)` void
- `assertSame(Object expected, Object actual, String message)` void
- `assertSame(Object expected, Object actual, Supplier<String> messageSupplier)` void

At the bottom of the dropdown, it says "Press ↵ to insert, ⇧ to replace".

The bottom status bar shows the project path: "CalculatorMavenProject > src > test > java > org > example > CalculatorTest". The right side of the status bar shows "6:48 LF UTF-8 4 spaces".

Assertions for Exception Handling



assertThrows

Verifies that the method under test throws an expected exception

Ensures the code handles invalid inputs or error conditions properly



assertDoesNotThrow

Confirms that a method runs without throwing exceptions

Excellent for validating stable behavior under normal input conditions

Exception-related assertions strengthen robustness and regression safety

Assertions for Exception Handling (contd.)

assertThrows

The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The main editor displays the file 'CalculatorTest.java' with the following code:

```
1 package org.example;
2
3 import org.example.org.example.Calculator;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.assertThrows;
7
8 public class CalculatorTest {
9
10     @Test
11     public void testPerformIntegerDivision() {
12         Calculator calculator = new Calculator();
13         int result = calculator.performIntegerDivision(10, 2);
14         assertEquals(5, result);
15     }
16 }
```

A code completion popup is visible for the `assertThrows` method, showing several overloads. The first two are highlighted:

- `assertThrows(Class<T> expectedType, Executable executable)` T
- `assertThrows(Class<T> expectedType, Executable executable, String message)` T

The bottom of the IDE shows the 'Run' tab with the command `CalculatorTest.testPerformIntegerDivision`. The output indicates that the test passed successfully:

```
Run CalculatorTest.testPerformIntegerDivision
✓ CalculatorTest (org.example) 14 ms
✓ 1 test passed 1 test
✓ testPerformIntegerDivision() 14 ms
Process finished with exit code 0
```

The status bar at the bottom shows the file path: `CalculatorMavenProject > src > test > java > org > example > CalculatorTest`, along with encoding and formatting settings: `6:57 LF UTF-8 4 spaces`.

Assertions for Exception Handling (contd.)

assertDoesNotThrow

The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The active file is 'CalculatorTest.java'. The code defines a package, imports the 'Calculator' class and JUnit 5's 'Test' and 'Assertions' classes, and then defines a test class 'CalculatorTest' with a test method 'testPerformIntegerDivision'. The test method uses 'assertDoesNotThrow' to verify that a division operation does not throw an exception. A tooltip is visible over the 'assertDoesNotThrow' method call, showing its various overloads. Below the code editor, the 'Run' tab shows the test execution results: '1 test passed' in 14 ms. The output console shows the command used to run the test and the message 'Process finished with exit code 0'.

```
1 package org.example;
2
3 import org.example.org.example.Calculator;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.assertDoesNotThrow; Tab to complete
7
8 public class CalculatorTest {
9
10     @Test
11     public void testPerformIntegerDivision() {
12         Calculator calculator = new Calculator();
13         int result = calculator.performIntegerDivision(10, 2);
14         assertDoesNotThrow(() -> result);
15     }
16 }
```

Run CalculatorTest.testPerformIntegerDivision

✓ CalculatorTest (org.example) 14 ms
✓ testPerformIntegerDivision() 14 ms

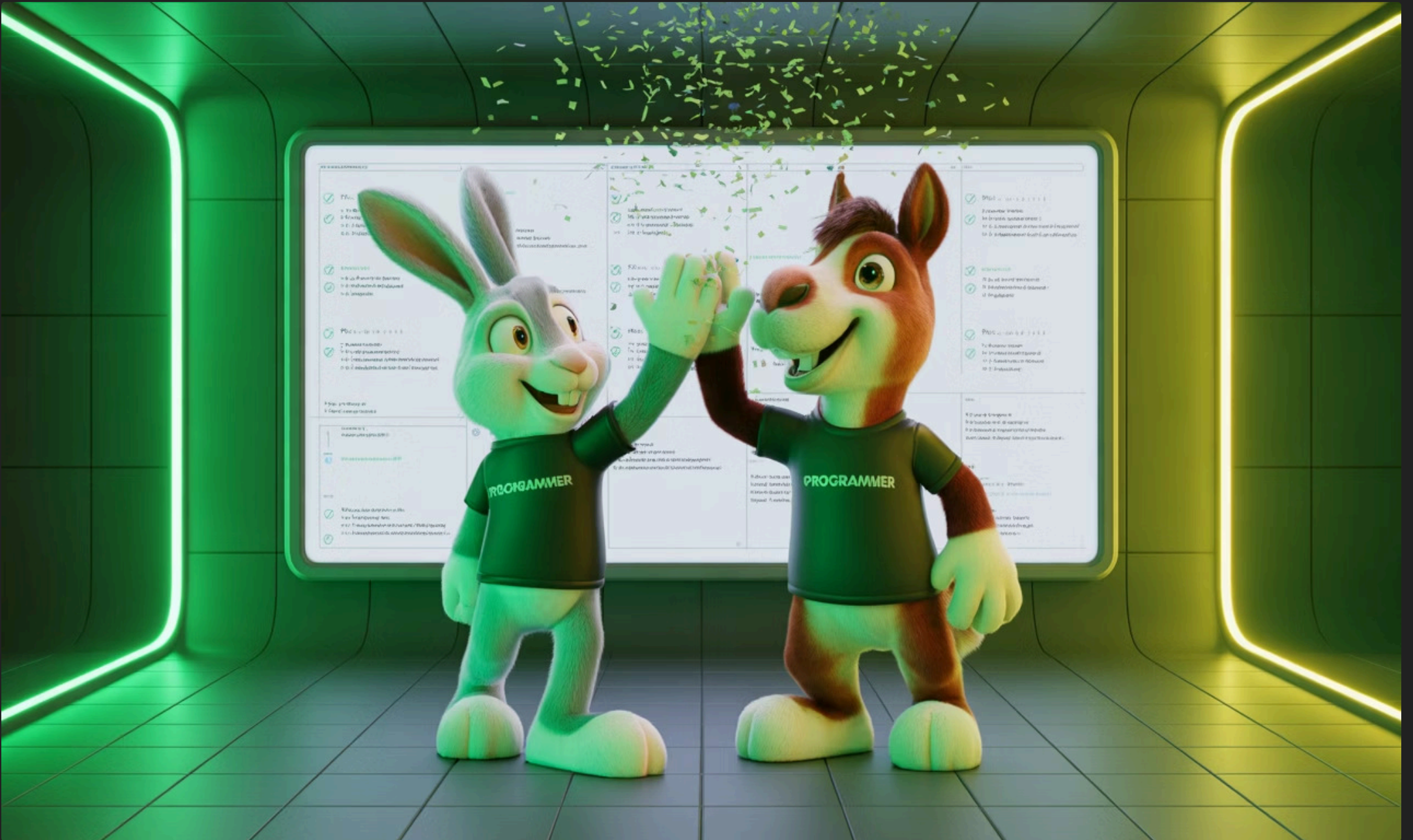
✓ 1 test passed 1 test total, 14 ms

/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...

Process finished with exit code 0

CalculatorMavenProject > src > test > java > org > example > CalculatorTest 6:55 LF UTF-8 4 spaces

Conclusion



JUnit provides a rich set of assertions tailored for different testing needs

Choosing the correct assertion improves clarity and test reliability

Assertions communicate intent and help developers quickly understand failures

Mastering assertions is essential for writing high-quality unit tests



Open Floor Discussion

Which assertions do you use most frequently in your projects?

How do you decide when to use `assertThrows` versus handling exceptions manually?

Do you prefer importing all assertions or only the ones you need?